Cryptography Theory & Practice

CIS 4930/6371 (Fall 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Sean Hernandez***
Contributor: ***Lazar Lazarevic***

October 2026

Homework 5: ECC-BAF (Due: *November* 2nd, 2026−11 : 59 *pm*)[100 Points]

(Upload your solutions to the CANVAS)
Requirements: Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

Introduction

Forward-secure digital signatures are cryptographic schemes that protect the integrity of past signatures even if the current signing key is compromised. This property is crucial for long-term security in applications where keys might be exposed over time. In this project, you will implement an Elliptic Curve-based Blind-Aggregate-Forward (ECC-BAF) scheme that provides forward security through key evolution.

The ECC-BAF scheme combines the efficiency of elliptic curve cryptography with forward security properties. Unlike traditional signature schemes where a single key compromise can invalidate all signatures, ECC-BAF evolves the private key state after each signing operation, ensuring that past signatures remain secure even if future keys are compromised.

This project involves implementing partial ECC-BAF scheme (excluding key generation), including signing with state updates, and verification procedures. You will work with elliptic curve operations, hash chains for key evolution, and cryptographic testing frameworks.

The project is divided into following components:

1. Investigate and understand the *ECC-BAF Key Generation* algorithm that creates private and public vectors
2. Implementation of the *ECC-BAF Update* procedures that evolve the private key state
3. Implementation of the *ECC-BAF Signing* procedures that sign messages
4. Implementation of the *ECC-BAF Verification* algorithm that validates signatures using public information

1 ECC-BAF Scheme

The ECC-BAF scheme operates on elliptic curves and uses hash chains to achieve forward security. The following algorithms define the complete scheme:

Algorithm 1 ECC-BAF Key Generation[YOU DO NOT NEED TO IMPLEMENT KEY GENERATION]

Require: Seed seed $\in \{0,1\}^{256}$, curve $\mathcal{E}$, base point G, order n, parameter L
Ensure: Public keys (A, B, u, u'), private state (a_0, b_0, x, x')

rnd $\leftarrow$ PRNG(seed, 32)
$a_0 \leftarrow H(\text{rnd} \| 0x01) \bmod n$
$b_0 \leftarrow H(\text{rnd} \| 0x02) \bmod n$
$x \leftarrow H(\text{rnd} \| 0x03) \bmod n$
$x' \leftarrow H(\text{rnd} \| 0x04) \bmod n$
// Build hash chains
$a[0] \leftarrow a_0, \quad b[0] \leftarrow b_0$
for $j = 0$ to $L - 2$ **do**
    $a[j+1] \leftarrow H(a[j]) \bmod n$
    $b[j+1] \leftarrow H(b[j]) \bmod n$
end for
// Compute public points
for $j = 0$ to $L - 1$ **do**
    $A[j] \leftarrow [a[j]]G$
    $B[j] \leftarrow [b[j]]G$
end for
// Compute auxiliary values
for $j = 0$ to $L - 1$ **do**
    $r[j] \leftarrow H(x \| j) \bmod n$
    $k[j] \leftarrow H(x' \| j) \bmod n$
    $u[j] \leftarrow (k[j] + r[j]) \bmod n$
    if $j \geq 1$ **then**
        $u'[j] \leftarrow (k[j-1] + H(k[j])) \bmod n$
    else
        $u'[j] \leftarrow \perp$
    end if
end for
return $(A, B, u, u'), (a_0, b_0, x, x')$

Algorithm 2 ECC-BAF Update

Require: Current index j, current secrets $(a_{\text{cur}}, b_{\text{cur}})$, order n
Ensure: Updated index $j+1$, updated secrets $(a'_{\text{cur}}, b'_{\text{cur}})$

$a_{\text{next}} \leftarrow H(a_{\text{cur}}) \bmod n$
$b_{\text{next}} \leftarrow H(b_{\text{cur}}) \bmod n$
secure_erase$(a_{\text{cur}}, b_{\text{cur}})$
$a_{\text{cur}} \leftarrow a_{\text{next}}$
$b_{\text{cur}} \leftarrow b_{\text{next}}$
$j \leftarrow j + 1$
return $(j, a_{\text{cur}}, b_{\text{cur}})$

Algorithm 3 ECC-BAF Signing

Require: Messages $D = \{D_0, D_1, \ldots, D_\ell\}$, current state $(j, a_{\text{cur}}, b_{\text{cur}})$, secrets (x, x'), order n
Ensure: Signature (σ, k_ℓ) where $\sigma \in \mathbb{Z}_n$ and $k_\ell \in \mathbb{Z}_n$

$\sigma \leftarrow 0$
for $m = j$ to ℓ **do**
  $r_m \leftarrow H(x \| m) \bmod n$
  $h_m \leftarrow H(D_m \| r_m \| m) \bmod n$
  $\text{term} \leftarrow (a_{\text{cur}} \cdot h_m + b_{\text{cur}}) \bmod n$
  $\sigma \leftarrow (\sigma + \text{term}) \bmod n$
  $(j, a_{\text{cur}}, b_{\text{cur}}) \leftarrow \text{Update}(j, a_{\text{cur}}, b_{\text{cur}}, n)$
end for
$k_\ell \leftarrow H(x' \| \ell) \bmod n$
return (σ, k_ℓ)

Algorithm 4 ECC-BAF Verification

Require: Messages $D = \{D_0, D_1, \ldots, D_\ell\}$, signature (σ, k_ℓ), public keys (A, B, u, u'), curve parameters $(\mathcal{E}, G, n)$
Ensure: `Accept/Reject`

// **Reconstruct** k **values**
$k_{\text{rec}}[\ell] \leftarrow k_\ell$
for $j = \ell$ down to 1 **do**
  $H_{k_j} \leftarrow H(k_{\text{rec}}[j]) \bmod n$
  $k_{\text{rec}}[j-1] \leftarrow (u'[j] - H_{k_j}) \bmod n$
end for
// **Reconstruct** r **values**
for $j = 0$ to ℓ **do**
  $r_{\text{rec}}[j] \leftarrow (u[j] - k_{\text{rec}}[j]) \bmod n$
end for
// **Compute right-hand side**
RHS $\leftarrow \mathcal{O}$ {Point at infinity}
for $j = 0$ to ℓ **do**
  $h_j \leftarrow H(D_j \| r_{\text{rec}}[j] \| j) \bmod n$
  temp $\leftarrow [h_j]A[j] + B[j]$
  RHS $\leftarrow$ RHS + temp
end for
// **Compute left-hand side**
LHS $\leftarrow [\sigma]G$
// **Verify equation**
if LHS = RHS **then**
  return `Accept`
else
  return `Reject`
end if

2 BAF vs ECC-BAF: Comparative Analysis [25 pt]

2.1 Traditional BAF (DLP-Based) vs ECC-BAF

The Blind-Aggregate-Forward (BAF) scheme was originally proposed using traditional discrete logarithm problem (DLP) in multiplicative groups. The ECC-BAF variant adapts these concepts to elliptic curve groups, providing advantages in terms of efficiency and security.

2.1.1 Mathematical Foundations

Traditional BAF (DLP-Based):

- **Group Structure:** Operates in multiplicative group $\mathbb{Z}_p^*$ where p is a large prime
- **Generator:** Uses generator g of a subgroup of order q in $\mathbb{Z}_p^*$
- **Hard Problem:** Security based on discrete logarithm problem in $\mathbb{Z}_p^*$
- **Key Sizes:** Requires 2048-3072 bit modulus for equivalent security
- **Operations:** Modular exponentiation: $g^x \bmod p$

ECC-BAF (Elliptic Curve Based):

- **Group Structure:** Operates in elliptic curve group $E(\mathbb{F}_p)$ or $E(\mathbb{F}_{2^m})$
- **Generator:** Uses base point G on elliptic curve with order n
- **Hard Problem:** Security based on elliptic curve discrete logarithm problem (ECDLP)
- **Key Sizes:** Achieves equivalent security with 256-384 bit keys
- **Operations:** Scalar multiplication: $[x]G$ (point multiplication)

2.1.2 Algorithmic Differences

Important Note: The signature computation σ is *identical* in both schemes - it consists of scalar arithmetic in the respective finite fields. The fundamental difference lies in how this scalar signature is *used* during verification:

- **Traditional BAF:** Uses σ as an *exponent* in multiplicative group operations
- **ECC-BAF:** Uses σ as a *scalar multiplier* in elliptic curve point operations

Security Foundation Clarification:

- The *signature value* σ itself is not "DLP-based" or "ECC-based" - it's pure modular arithmetic
- The *security* comes from the hardness of inverting the verification equation:
  - Traditional BAF: Hard to find σ such that $g^\sigma = \text{target}$ (DLP)
  - ECC-BAF: Hard to find σ such that $[\sigma]G = \text{target point}$ (ECDLP)
- Both schemes use the same forward-secure key evolution mechanism

Component	Traditional BAF (DLP)	ECC-BAF
Public Keys	$A[j] = g^{a[j]} \bmod p$ $B[j] = g^{b[j]} \bmod p$	$A[j] = [a[j]]G$ $B[j] = [b[j]]G$
Key Evolution	$a_{j+1} = H(a_j) \bmod q$ $b_{j+1} = H(b_j) \bmod q$	$a_{j+1} = H(a_j) \bmod n$ $b_{j+1} = H(b_j) \bmod n$
Signature Computation	**Identical scalar arithmetic:** $\sigma = \sum_{m=j}^{\ell}(a_m \cdot h_m + b_m) \bmod \text{order}$	
Verification **(Key Difference)**	$g^{\sigma} \stackrel{?}{=} \prod_{j=0}^{\ell}(A[j]^{h_j} \cdot B[j]) \bmod p$ *Exponentiation with* σ	$[\sigma]G \stackrel{?}{=} \sum_{j=0}^{\ell}([h_j]A[j] + B[j])$ *Scalar multiplication by* σ
Group Law	Multiplication in $\mathbb{Z}_p^*$	Addition in elliptic curve group

Table 1: Comparison of mathematical operations in BAF vs ECC-BAF

2.1.3 Performance Analysis

Computational Efficiency:

- **Key Generation:** ECC-BAF requires smaller key generation time due to shorter parameters
- **Signing:** ECC-BAF scalar multiplication is generally faster than modular exponentiation
- **Verification:** ECC point operations can be optimized using precomputation techniques
- **Memory Usage:** ECC-BAF uses significantly less memory for key storage

Communication Overhead:

- **Traditional BAF:** Public keys and signatures require 2048-3072 bits each
- **ECC-BAF:** Equivalent security with 256-384 bit representations
- **Bandwidth Reduction:** ECC-BAF achieves 6-8x reduction in data transmission

2.1.4 Security Considerations

Security Equivalence:

Security Level	DLP Key Size	ECC Key Size
80 bits	1024 bits	160 bits
112 bits	2048 bits	224 bits
128 bits	3072 bits	256 bits
192 bits	7680 bits	384 bits
256 bits	15360 bits	512 bits

Forward Security Properties: Both BAF and ECC-BAF provide identical forward security guarantees:

- **Key Evolution:** Irreversible hash chain progression prevents backward computation
- **State Updates:** Secure erasure of old keys ensures forward security
- **Signature Validity:** Past signatures remain valid even after key compromise

3 Implementation Requirements

3.1 ECC-BAF Sign and Update Implementation [50 pt]

Input Arguments

- ECC curve parameters file **"ECCParams.txt"**
- Message file **"Message.txt"**
- Initial private state file **"InitialPrivateState.txt"**

The signing program performs the following steps:

1. Read elliptic curve parameters (curve name: secp256r1, secp256k1, or secp384r1)
2. Read message(s) from the message file
3. Read current private state: $(cur_j, a_cur, b_cur, x, x')$
4. Execute the ECC-BAF signing algorithm for messages $D_j, D_{j+1}, \ldots, D_\ell$
5. Update private state through hash chain evolution
6. Output signature (σ, k_ℓ) to **"signature.txt"**
7. Save updated private state to **"private_state.txt"**

Terminal:

```
Compile the signing part with the following command:
gcc ecc_baf_sign_and_update.c -lcrypto -o ecc_baf_sign_and_update
Run your code with the following commands:
./ecc_baf_sign_and_update ECCParams.txt Message.txt InitialPrivateState.txt
```

3.2 ECC-BAF Verification Implementation [50 pt]

Input Arguments

- ECC curve parameters file **"ECCParams.txt"**
- Message file **"Message.txt"**
- Public vector file **"PublicVector.txt"**

The verification program performs the following steps:

1. Read elliptic curve parameters
2. Read message(s) from the message file
3. Reconstruct K values and R values from the u and u' vectors
4. Read public vectors: $A[0..L-1], B[0..L-1], u[0..L-1], u'[0..L-1]$
5. Read signature (σ, k_ℓ) from **"signature.txt"**

6. Execute the ECC-BAF verification algorithm
7. Output verification result to **"verify_result.txt"** (VALID/INVALID)

Terminal:

```
Compile the verification part with the following command:
gcc ecc_baf_verify.c -lcrypto -o ecc_baf_verify
Run your code with the following commands:
./ecc_baf_verify ECCParams.txt Message.txt PublicVector.txt
```

4 Implementation Architecture

Your ECC-BAF implementation uses the following file structure and data flow:

File Type	Purpose	Format
ECCParams{1,2,3}.txt	Curve specifications	curve: secp256r1/secp256k1/secp384r1
Message{1,2,3}.txt	Test messages	Plain text message content
PublicVector{1,2,3}.txt	Public key material	A[i], B[i], u[i], up[i] arrays
InitialPrivateState{1,2,3}.txt	Private key state	cur_j, a_cur, b_cur, x, xp values
signature.txt	ECC-BAF signature	σ and k_ℓ values
verify_result.txt	Verification output	VALID or INVALID

Table 2: File structure for ECC-BAF implementation

Key Security Properties

The ECC-BAF scheme provides the following security guarantees:

1. **Forward Security:** Compromise of current keys does not affect the validity of past signatures
2. **Unforgeability:** Signatures cannot be forged without knowledge of the current private key
3. **Efficiency:** Elliptic curve operations provide security with smaller key sizes
4. **Key Evolution:** Hash chains ensure irreversible key updates

Cryptographic Components

Your implementation relies on the following cryptographic primitives:

- **Elliptic Curve Operations:** Point addition and scalar multiplication
- **Hash Functions:** SHA-256 for key evolution and message hashing
- **Modular Arithmetic:** Operations in the finite field $\mathbb{Z}_n$
- **Secure Random Generation:** PRNG for initial key material

Notes

- **Refer to the OpenSSL and ECC tutorials for details on how to use the PRNG function, BIGNUM and EC operations.**
- You can also refer to the OpenSSL library documentation https://docs.openssl.org/master/ to learn how to use the BIGNUM and EC operations (It does not cover the PRNG).
- In grading on our side, we will test your code with multiple different seeds and messages, and also with a wrong signature. Hence, make sure your program does as instructed.
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn't matter if it works on your computer!).
- Compile and run your code for functionality with provided commands. You can check your code for correctness with the script to verify your solution for the whole programming project. Please, only submit your code if it is functional.
- In order to use the script, just put your codes, the rest of the provide files along with the "RunTestVectors.sh" in one folder and run the following command in the terminal: ./RunTestVectors.sh
- **Deliverables:** `ecc_baf_sign_and_update.c`, `ecc_baf_verify.c`

5 Rubric

- **ECC-BAF Sign and Update Implementation [50 pt]**
  - Correct "`private_state.txt`" generation **[15 pt]**
  - Correct "`signature.txt`" generation **[35 pt]**
- **ECC-BAF Verify Implementation [50 pt]**
  - Correct "`reconstructed_k.txt`" generation **[15 pt]**
  - Correct "`reconstructed_r.txt`" generation **[15 pt]**
  - Correct "`verify_result.txt`" generation **[20 pt]**